

Estado da arte e delimitação de originalidade

Fábrica de software multi-agente — análise comparativa

Enzo Consulo · 29 de agosto de 2026 · repositório público: github.com/enzoconsulo/Generic-Multi-Agents

Este documento responde a três perguntas: o que já existe que se parece com este trabalho, onde há sobreposição real, e o que o distingue. A conclusão, adiantada: **o trabalho não é plágio, mas vários de seus componentes não são originais** — e a diferença entre honestidade acadêmica e problema é citar, não ser inédito. Um trabalho de conclusão não precisa inventar; precisa dizer com precisão o que herdou e o que acrescentou.

1. O que existe

1.1 Agentic Software Factory (softwarefabrik.io)

Produto comercial, de Martin Janda, versão v0.30.0 (2026). É o projeto mais próximo, e a semelhança de vocabulário é real: linha de produção estruturada, portão de qualidade, disciplina de git, modelo por papel. Opera em três fases — assistente, execução e **sign-off**. A entrada é um assistente de seis passos com dezoito modelos de stack; cada execução roda em contêiner Docker efêmero, com sistema de arquivos somente leitura e sem rede; o portão final tem seis papéis revisores.

A DIVERGÊNCIA ESTRUTURAL O produto rejeita explicitamente a autonomia: há aprovação humana, com exibição das diferenças, antes de cada passo. Este trabalho vai na direção oposta — uma única intervenção humana, no pedido inicial, e os portões são operados por agentes. São respostas opostas à mesma pergunta, e a autonomia é o problema mais difícil dos dois.

1.2 A literatura acadêmica

ChatDev (arXiv:2307.07924) e MetaGPT, ambos de 2023, são a origem acadêmica da ideia de papéis especializados de agentes produzindo software. O ChatDev organiza papéis de empresa — diretor executivo, diretor técnico, programador, testador, revisor — num ciclo em cascata, com cada fase conduzida por duplas de agentes em diálogo. O MetaGPT usa a mesma metáfora, com procedimentos operacionais padrão codificados. **Qualquer banca que conheça a área cobrará a citação destes dois.**

1.3 Frameworks de agentes em Elixir/OTP

Este é o achado mais desconfortável, e precisa ser dito: **agente como processo supervisionado não é uma ideia original deste trabalho**. É prática estabelecida na comunidade Elixir, com implementações públicas — Cortex (orquestração multi-agente com fluxos em grafo acíclico e painéis ao vivo), Sagents (supervisão OTP, delegação a subagentes, painel LiveView), SwarmEx e Synapse (com persistência em PostgreSQL). O argumento da escolha de plataforma está correto, mas não é inédito, e apresentá-lo como descoberta seria um erro que um avaliador da área percebe.

1.4 O campo em 2026, e a origem do termo

A Factory.ai posiciona-se comercialmente como fábrica de software agêntica; em repositórios públicos há, entre outros, The Dark Factory, Galley, Prestige e Lorg. E o termo em si não é da era da inteligência artificial: vem da engenharia de software das décadas de 1960 a 1990, notadamente de Cusumano, Japan's Software Factories (1991), que descreve linhas de produção com processo padronizado e reúso.

2. Onde há sobreposição real

Sem eufemismo. Cada linha abaixo é algo que este trabalho tem e outros também têm.

Mecanismo	Quem já tem
Termo “fábrica de software” e a metáfora da linha de produção	Cusumano (1991); softwarefabrik; Factory.ai
Agentes com papéis especializados produzindo software	ChatDev, MetaGPT (2023)
Portão de qualidade antes de integrar	softwarefabrik; Galley; Dark Factory
Modelo mais forte para uns papéis, mais barato para outros	softwarefabrik
Um commit por unidade de trabalho	softwarefabrik; Dark Factory
Estimativa de custo por execução	softwarefabrik
Agente como processo supervisionado (OTP)	Cortex, Sagents, SwarmEx, Synapse
Painel ao vivo acompanhando agentes	Cortex, Sagents
Dirigir a ferramenta de linha de comando por arquivos de papel	softwarefabrik (convergência de substrato)
Fila durável com persistência em PostgreSQL	Synapse

São dez itens. Um trabalho que apresentasse os dez como contribuição própria estaria em apuros — não por plágio, mas por falta de revisão da literatura.

3. O que distingue este trabalho

3.1 A trilha não-software, com escada de prova

É a parte mais provavelmente original: não foi encontrado equivalente publicado. O eixo que separa os dois casos não é “é código?”, e sim **como se prova que ficou pronto**. Em software a prova vem de graça — o programa roda e passa ou quebra. Fora dele não vem, e é aí que o desenho degeneraria, com o verificador virando um segundo revisor e dois julgamentos subjetivos custando em dobro.

A solução tem três peças: todo critério fica em um de três degraus — executado, inspecionado ou julgado — e o verificador é **obrigado a rotular** qual usou; a primeira tarefa instala o verificador, antes de qualquer parte do artefato; e a fonte é texto versionado, com o binário gerado por comando, porque versionar o binário como fonte apagaria o portão da revisão. A proporção de critérios apenas julgados fica visível por projeto: quando ela sobe, o sistema está operando com um portão e meio, e isso vira sinal de replanejamento.

REFERÊNCIA Commit 2255150 (01/08) cria a trilha genérica; 9a1c186 (14/08) estende o rótulo de grau de prova também à trilha de software.

3.2 Reprojetado guiado pela telemetria da própria operação

Esta é a contribuição de pesquisa, e a que nenhum outro projeto pode ter, porque depende de dados que só existem neste repositório. A primeira versão rodou seis semanas em uso real, e a segunda é reprojetada a partir do que foi medido: 93,14% dos tokens de entrada são leitura de memória temporária do provedor, e a escrita, com 6,66% dos tokens, carrega cerca de metade da conta; 78% do custo estava em sessões do coordenador fazendo trabalho de máquina de estados. **Nenhum dos projetos pesquisados publica medição da própria operação** — os acadêmicos medem qualidade de saída em conjuntos de referência; o comercial estima custo antes de executar, e não mede o gasto acumulado para reprojetar.

3.3 Dois portões que fazem perguntas diferentes

Não são dois níveis de zelo. O verificador pergunta “funciona?” e executa os critérios; o revisor pergunta “é o que foi pedido?” e “está correto?”, em seções separadas. O ponto não-óbvio: **uma entrega pode passar em todos os critérios, não ter defeito nenhum, e ainda assim não ser a tarefa**, quando o critério foi mal escrito. Reprovar por

não-conformidade não exige encontrar defeito. Não foi encontrada essa separação articulada nos projetos pesquisados.

3.4 O coordenador não é um agente

No ChatDev e no MetaGPT a coordenação acontece por diálogo entre agentes; na softwarefabrik, quem coordena é o humano. Aqui, quem decide ordem, promoção, roteamento e escalonamento é código determinístico. E a razão não foi estética: mediu-se que o coordenador baseado em modelo consumia trinta e seis idas ao modelo e cerca de 17% do custo de um trabalho para fazer o que é, literalmente, uma máquina de estados. A fronteira que ficou: **cabe num teste? então é código**. Replanejar, julgar um marco reprovado e redigir relatório continuam no modelo, porque são julgamento.

REFERÊNCIA Commits 91559b8 (02/08), que converte o laço em código testado, e 8017c41 (02/08), cuja mensagem registra: “o pipeline decide sozinho — sem portão de julgamento humano”.

3.5 Autoridade como ausência de ferramenta

“Quem revisa não corrige” é regra em vários projetos. Aqui o revisor não recebe a ferramenta de escrever: a regra deixa de ser interpretável e vira impossibilidade, verificada por um teste que percorre o catálogo de ferramentas de cada papel. Restringir ferramentas por papel é prática conhecida; o que parece próprio é usá-la como o mecanismo de separação de autoridade, e testá-la negativamente.

3.6 A escada de resposta ao fracasso

Quatro degraus, cada um disparado por fato registrado: a primeira reprovação sobe de modelo; a segunda sob o mesmo especialista troca de especialista; três ciclos esgotados replanejam, com o planejador quebrando a tarefa em duas ou três menores; e, esgotando de novo, bloqueia. Limite de tentativas e escalonamento de modelo são comuns. O terceiro degrau é o incomum: reconhecer que o problema pode ser de dimensionamento da tarefa, não de execução, e agir sobre isso automaticamente.

3.7 Criação sob demanda

Nada é pré-fabricado. De uma frase em linguagem natural saem especificação, plano em fases e de oito a vinte tarefas com dependências declaradas; a equipe é sintetizada em dois a cinco especialistas a partir do próprio pedido; e quando o pedido não cabe nas categorias existentes, o planejador cunha o domínio, declarando o trio artefato, geração e verificação. O contraste é direto: a softwarefabrik oferece dezoito modelos de stack, e ChatDev e MetaGPT têm elenco fixo. Aqui não há modelo nem elenco — há síntese.

4. Comparação lado a lado

Eixo	Este trabalho	softwarefabrik.io	ChatDev / MetaGPT
Quem aprova	Agentes, sem a ferramenta de escrever	O humano, antes de cada passo	Agentes revisores, em diálogo
Quem coordena	Código determinístico	O humano, pela interface	Diálogo entre agentes
Intervenção humana	Uma só: o pedido inicial	Por aprovação; autonomia rejeitada	Pedido inicial
Origem do projeto	Sob demanda, sintetizada do pedido	Dezoito modelos num assistente	Elenco fixo de papéis
Isolamento em paralelo	Arquivos declarados, verificados pelo motor	Contêiner Docker efêmero	Não trata
Ao falhar	Escada de quatro degraus, com replanejamento	O humano decide	Novo ciclo de diálogo
Artefatos não-software	Trilha própria, com escada de prova	Não trata	Não trata
Base do desenho	Telemetria própria: 139 execuções	Experiência do autor	Conjuntos de referência

	medidas		
--	---------	--	--

5. Evidência documental

O repositório é público e a verificação é direta. Entre 18 de julho e 28 de agosto de 2026 acumulou 205 commits, e os conceitos aparecem **em sequência, depois dos incidentes que os motivaram**, não prontos no primeiro dia. Um projeto transcrito chega inteiro; este chegou por acúmulo.

Commit	Data	O que registra
8215dbf	01/08	I1 — MAPA.md, índice denso do projeto no lugar da varredura de arquivos
9d43b61	01/08	doutrina de custo de contexto: o modelo, a medição e as cinco intervenções
2255150	01/08	trilha genérica — a fábrica deixa de ser só de software
f5b8fd4	01/08	limiar medido de tamanho de tarefa, no planejador e antes do despacho
1a41ca8	02/08	montador de contexto, máquina do pipeline e teto de custo com parada limpa
4c559ef	02/08	I2 (prefixo cacheável) e I5 (critérios executáveis)
91559b8	02/08	I3 — o laço de orquestração vira código testado
8017c41	02/08	o pipeline decide sozinho — sem portão de julgamento humano
cdedec2	02/08	saneamento passa a olhar a árvore git (defeito achado em rodada real)
9a1c186	14/08	verificador passa a declarar o grau de prova, nas duas trilhas
eb9e51f	16/08	prova que a equipe especializada chega ao modelo, e corrige a doutrina
e8974c5	21/08	portão do meio avaliado e fechado — 0 s de fila em 43 rodadas

Medições que contrariaram a própria hipótese

O sinal de que o método é honesto não é a medição que confirma o que já se supunha, e sim a que obriga a mudar de rumo. Quatro casos, todos registrados em commit:

- **174f0a0 (31/07) — previsões escritas antes da rodada**, para poder confrontá-las com o resultado depois.
- **696762e (31/07) — “/status remedido (n=3): o ganho de 12% era ruído”**. Um resultado anunciado foi refutado por nova medição, e retirado.
- **dea9e9c (31/07) — a contagem de saída estava cem vezes subestimada**. O instrumento estava errado, e foi corrigido antes de qualquer conclusão.
- **e8974c5 (21/08) — hipótese avaliada e descartada por medição**: supunha-se que o portão de verificação seria o gargalo do sistema; medido sobre quarenta e três rodadas, houve zero segundo de espera. A otimização já desenhada foi abandonada por evidência.

Operação real

A primeira versão entregou 86 tarefas concluídas de 89, em três projetos de domínios distintos: um jogo de tabuleiro em versão web, multijogador em tempo real (69 de 72 tarefas); um serviço permanente embarcado no computador de placa única que controla uma impressora 3D (11 de 11); e uma plataforma de dados com camada de análise por modelos de linguagem (6 de 6). São 139 execuções gravadas, com tokens, custo, modelo e desfecho, preservadas no repositório.

6. Conclusão e encaminhamento

Não é plágio, e a evidência documental sustenta isso. Mas o trabalho não é inédito no gênero, e não deve se apresentar como se fosse. **Plágio é sobre atribuição, não sobre ineditismo.**

Três encaminhamentos:

- **Escrever a seção de trabalhos relacionados e citar antes de ser perguntado:** ChatDev e MetaGPT pelos papéis; softwarefabrik.io e Factory.ai pela fábrica agêntica comercial; Cortex, Sagents, SwarmEx e Synapse pelo agente como processo supervisionado; Cusumano pelo termo.
- **Reduzir a afirmação de contribuição ao escopo real.** Não “construí uma fábrica de software multi-agente” — isso existe. É “operei uma por seis semanas em três projetos reais, medi onde ela falha e quanto custa, e reprojetei a partir dessa medição”.
- **Comparar explicitamente com o vizinho mais próximo.** Demonstrar que se conhece o projeto mais parecido e que se escolheu diferente, com motivo, é a defesa mais sólida disponível.

Um trabalho que declara isso com precisão é mais forte do que um que reivindica ineditismo total.

Fontes consultadas

- softwarefabrik.io — Agentic Software Factory (Martin Janda), v0.30.0, 2026
- arXiv:2307.07924 — ChatDev: Communicative Agents for Software Development, 2023
- factory.ai — Factory 2.0: from coding agents to software factories
- github.com/itsHabib/cortex — orquestração multi-agente em Elixir/OTP
- github.com/sagents-ai/sagents — agentes com supervisão OTP e painel ao vivo
- CUSUMANO, M. Japan's Software Factories. Oxford University Press, 1991